

A) Clean code development

1. The Destination class defines fields like country, city, weather, budget, and activities, which are intuitive and self-explanatory. These names clearly represent the purpose of each field, reducing the need for additional comments or explanations.

```
public class Destination {  
    private String country;  
    private String city;  
    private String weather;  
    private String budget;  
    private List<String> activities;
```

2. In the Destination class, the fields are private, and getters/setters are used to access them. This ensures that the internal state of the class is protected, and any changes are controlled through the setter methods, adhering to object-oriented principles.

```
public String getCountry() {  
    return country;  
}  
public String getCity() {  
    return city;  
}  
public String getWeather() {  
    return weather;  
}  
public String getBudget() {  
    return budget;  
}  
public List<String> getActivities() {  
    return activities;  
}  
  
public String toString() {  
    return "Destination: " + city + ", " + country + "\nWeather: " + weather + "\nBudget: " + budget + "\nActivities: " + ac  
}
```

3. Both NextDestination.java and Destination.java files follow consistent indentation and spacing conventions. Consistent formatting improves readability, making it easier for developers to navigate and understand the codebase.
4. The Destination class is solely responsible for holding destination data, while logic for filtering and processing is handled separately in NextDestination.java.

```

private List<Destination> findMatches(String weather, String budget, List<String> activities) {
    List<Destination> matches = new ArrayList<>();

    for (Destination dest : destinations) {
        if (dest.getWeather().equals(weather) && dest.getBudget().equals(budget)) {
            boolean allActivitiesPresent = true;
            for (String activity : activities) {
                if (!dest.getActivities().contains(activity)) {
                    allActivitiesPresent = false;
                    break;
                }
            }

            if (allActivitiesPresent) {
                matches.add(dest);
            }
        }
    }

    return matches;
}

```

5. The destinations are loaded dynamically from a JSON file (destinations.json) rather than hardcoding data in the program. This improves scalability and makes it easier to update data without modifying the source code.

```

private void loadDestinations() {
    try {
        FileReader reader = new FileReader(fileName:"destinations.json");
        Gson gson = new Gson();
        DestinationData data = gson.fromJson(reader, DestinationData.class);
        destinations = data.destinations;
    } catch (IOException e) {
        System.out.println("Error reading file: " + e.getMessage());
    }
}

```

B) Personal CCD cheat sheet

1. Names should clearly indicate their purpose.
2. Stick to consistent naming conventions.
3. Keep functions small and focused. Ideally, they should perform one task.
4. Use private fields with public getters and setters to control access.
5. Clean up unused variables, imports, or methods.
6. Break down complex logic into simpler, named methods or use helper functions.
7. Implement exception handling to ensure the app behaves predictably during runtime errors.
8. Write unit tests to verify the behavior of individual components.
9. Avoid deeply nested conditions. Use early returns or split logic into smaller functions.
10. Avoid duplicating code. Refactor shared logic into reusable methods.
11. Group related methods and classes logically.
12. Avoid unnecessary loops or difficult operations inside frequently called methods.
13. Leverage built-in libraries and frameworks instead of building solutions on your own.